RETROENGINE BETA ENCYCLOPEDIA

Volume 1: Developer & User Manual — Core Systems, ECS & Scripting Engine

1. EXECUTIVE OVERVIEW & PHILOSOPHY

RetroEngine is a modern, lightweight 3D game engine and editor suite built from the ground up on **Java 25**. Inspired by classic early-2000s engine architectures like RenderWare 3.5, Gamebryo, and the Creation Engine, RetroEngine combines architectural simplicity with cutting-edge multi-threaded platform capabilities including JDK Virtual Threads, zero-allocation hot paths, dynamic BVH spatial indexing, and dual Vulkan 1.3 / Metal 3 graphics backends.

The engine explicitly targets high extensibility and modding potential—drawing inspiration from Minecraft's modding ecosystem—by incorporating an in-memory Janino Java compiler (`SimpleCompiler`) for instant gameplay script hot-reloading without native C++ build delays.

2. CORE SYSTEMS & ARCHITECTURE TOPOLOGY

Subsystem	Implementation Class	Memory Strategy	Key Responsibility
Core Loop	**`EngineLoop` / `RuntimeLoop`**	Sub-ms Spin-Wait	60 FPS frame pacing, GLFW input polling, delta smoothing window.
ECS Storage	**`EntityManager` / `ComponentPool`**	Dense Array Packing	64-bit generational handles (index | gen), zero pointer-chasing.
Script Engine	**`ScriptEngine` / `EntityScript`**	Janino Dynamic Compiler	In-memory Java bytecode compilation, live source hot-reloading.
Asset Streamer	**`AssetStreamer`**	JDK 25 Virtual Threads	Asynchronous non-blocking disk I/O, Assimp model & ImageIO texture streaming.
Physics World	**`PhysicsSystem`**	60 Hz Sub-stepped Solvers	Dynamic BVH AABB tree broadphase, narrowphase manifolds, vehicle suspension.
Render Pipeline	**`SceneRenderPipeline`**	Thread-Safe Snapshots	View-frustum culling, skeletal bone palette extraction, RHI dispatch.
RHI Backends	**`VulkanBackend` / `MetalBackend`**	Handle-Based VRAM Tables	Vulkan 1.3 bindless descriptor tables, Metal 3 argument buffers.

Key Beta Architecture Rule: Zero-Allocation Hot Paths

All core tick loops (`PhysicsSystem`, `SceneRenderPipeline`, `GameUIRenderer`, `UIGeometryBatch`) reuse pre-allocated primitive arrays and pooled snapshot buffers. No JVM garbage-collection pauses occur during gameplay simulation.

3. PROJECT STRUCTURE & DISK-MIRRORED INGESTION

RetroEngine enforces a disk-mirrored project topology where the `Assets/` directory on disk serves as the single source of truth. All files, folders, and resources beneath `Assets/` are mirrored in memory by the `AssetDatabase`.

Directory / File	Format / Type	Role & Description
`ProjectSettings.bin`	Binary Header ('RPRJ')	Project metadata, engine version, and startup scene path.
`Assets/`	Disk Source Directory	Root folder for all user scenes (.rscene), scripts (.java), models, and textures.
`Cooked/`	Binary VRAM Packages	Pre-swizzled .rpack archives produced by offline or runtime asset baking.
`.retro/asset_registry.bin`	128-bit UUID Manifest	Centralized UUID manifest mapping 128-bit UUIDs to project-relative paths.

ASSET STREAMING & REFERENCE-TRACKED VRAM EVICTION

The `AssetStreamer` subsystem leverages **JDK 25 Virtual Threads** (`Executors.newVirtualThreadPerTaskExecutor()`) for non-blocking background Assimp 3D mesh parsing, ImageIO PNG/JPG decoding, and pre-cooked `.rpack` VRAM package loading. Streamed assets are uploaded to GPU VRAM directly on virtual worker threads and assigned opaque 64-bit handles.

To prevent memory leaks without requiring explicit manual allocation hooks, streamed assets are reference-tracked by frame stamps. The render thread calls `markMeshReferenced` every frame for active entities. Unreferenced assets are automatically swept and evicted from host RAM and GPU VRAM after a configurable grace period (`sweepUnreferenced`).

128-Bit UUID Indirection Standard

Scene files (`.rscene`) and prefabs store asset references as persistent 128-bit UUIDs rather than raw file paths. Moving or renaming files inside `Assets/` updates the `asset_registry.bin` manifest while preserving scene integrity.

4. IN-MEMORY SCRIPTING ENGINE & LIFECYCLE HOOKS

RetroEngine features an in-memory Java compilation engine powered by Janino (`SimpleCompiler`). Gameplay logic is authored by extending `EntityScript`. Script files stored under `Assets/` are parsed, compiled, and instantiated dynamically in memory at runtime.

Lifecycle Hook / Callback	Trigger Timing / Condition	Usage & Purpose
`onAwake()`	Entity Script Instantiation	Initialize non-spatial variables and data structures.
`onEnable()` / `onDisable()`	Component Activation Toggle	Register or unregister event listeners when component is toggled.
`onStart()`	First Frame of Play State	Query references to neighboring entities and setup initial transforms.
`onUpdate(float deltaTime)`	Every Render Frame Tick	Poll user input, execute state machine logic, update smooth movement.
`onLateUpdate(float dt)`	Post-Update Render Frame	Camera tracking, inverse kinematics, post-animation adjustments.
`onPhysicsUpdate(float fdt)`	Fixed 60 Hz Physics Tick	Apply forces, set velocities, drive rigidbodies and vehicle controllers.
`onCollisionEnter(Entity e)`	RigidBody Manifold Touch	Dispatched on first frame 3D physics colliders touch.
`onCollisionStay(Entity e)`	RigidBody Manifold Overlap	Dispatched every fixed frame 3D physics colliders remain in contact.
`onCollisionExit(Entity e)`	RigidBody Manifold Separate	Dispatched on frame 3D physics colliders separate.
`onTriggerEnter(Entity e)`	Trigger Volume Intersection	Dispatched when entity enters an `isTrigger` collider volume.
`onTriggerStay(Entity e)`	Trigger Volume Overlap	Dispatched every frame entity remains inside an `isTrigger` volume.
`onTriggerExit(Entity e)`	Trigger Volume Exit	Dispatched when entity exits an `isTrigger` collider volume.
`onMouseEnter()` / `Exit()`	Color-ID Hover Event	Dispatched when Viewport mouse cursor enters/leaves entity mesh bounds.
`onMouseDown()` / `Up()`	Color-ID Click Event	Dispatched when user clicks/releases mouse on entity mesh in 3D viewport.
`onDrawGizmos()`	Editor Render Frame Pass	Render custom debug lines, wireframe spheres, and raycasts in Editor.
`onDestroy()`	Entity Removal / Scene Unload	Release pooled resources, stop sound instances, clear handle references.

Hot-Loop Exception Suppression

If a user script throws an unhandled exception during `onUpdate()` or `onPhysicsUpdate()`, `ScriptEngine` catches the exception, logs a stack trace to the `EngineConsolePanel`, and disables that script instance. The core engine never crashes due to gameplay script bugs.

5. PRODUCTION GAMEPLAY SCRIPTS — EXAMPLE 1: PLAYER CONTROLLER

Below is a complete, production-ready Vice City style 3D character controller script demonstrating input polling, camera-relative movement direction calculation, jumping, and character controller solver interaction.

■ PlayerControllerScript.java

```
package com.retro.game;

import com.retro.engine.scripting.EntityScript;
import com.retro.engine.scripting.Input;
import com.retro.engine.scene.components.gameplay.CharacterControllerComponent;
import com.retro.engine.scene.components.media.AnimatorComponent;
import org.lwjgl.csrf.GLFW;

public class PlayerControllerScript extends EntityScript {
    public float moveSpeed = 6.0f;
    public float sprintMultiplier = 1.8f;
    public float jumpForce = 8.5f;

    private CharacterControllerComponent controller;
    private AnimatorComponent animator;

    @Override
    public void onStart() {
        controller = getCharacterController();
        animator = getAnimator();
    }

    @Override
    public void onPhysicsUpdate(float fixedDeltaTime) {
        if (controller == null) return;

        float moveX = 0.0f, moveZ = 0.0f;
        if (Input.isKeyDown(GLFW.GLFW_KEY_W)) moveZ += 1.0f;
        if (Input.isKeyDown(GLFW.GLFW_KEY_S)) moveZ -= 1.0f;
        if (Input.isKeyDown(GLFW.GLFW_KEY_A)) moveX -= 1.0f;
        if (Input.isKeyDown(GLFW.GLFW_KEY_D)) moveX += 1.0f;

        boolean isSprinting = Input.isKeyDown(GLFW.GLFW_KEY_LEFT_SHIFT);
        float currentSpeed = moveSpeed * (isSprinting ? sprintMultiplier : 1.0f);

        // Convert camera-relative direction to world space
        float[] fwd = getForward();
        float[] right = getRight();
        float dx = (fwd[0] * moveZ + right[0] * moveX) * currentSpeed;
        float dz = (fwd[2] * moveZ + right[2] * moveX) * currentSpeed;

        controller.walkVelocityX = dx;
        controller.walkVelocityZ = dz;

        if (Input.isKeyPressed(GLFW.GLFW_KEY_SPACE) && controller.isGrounded) {
            controller.jump(jumpForce);
        }

        if (animator != null) {
            float speedMag = (float) Math.sqrt(dx * dx + dz * dz);
            animator.setFloat("Speed", speedMag);
            animator.setBool("IsGrounded", controller.isGrounded);
        }
    }
}
```

PRODUCTION GAMEPLAY SCRIPTS — EXAMPLES 2 & 3

Example 2: Health Pickup Trigger Script

■ **HealthPickupScript.java**

```
package com.retro.game;

import com.retro.engine.scripting.EntityScript;
import com.retro.engine.scene.entity.Entity;
import com.retro.engine.scene.components.media.AudioSource;

public class HealthPickupScript extends EntityScript {
    public int healthAmount = 25;
    public float rotationSpeed = 90.0f;

    @Override
    public void onUpdate(float deltaTime) {
        rotate(0.0f, rotationSpeed * deltaTime, 0.0f);
    }

    @Override
    public void onTriggerEnter(Entity other) {
        if (other != null && other.name.equals("Player")) {
            AudioSource audio = getAudioSource();
            if (audio != null) audio.play();
            logInfo("Health pickup collected by " + other.name + " (+ " + healthAmount + " HP)");
            sceneWorld.destroyEntity(entity);
        }
    }
}
```

Example 3: Character Animator Driver Script

■ **CharacterAnimatorScript.java**

```
package com.retro.game;

import com.retro.engine.scripting.EntityScript;
import com.retro.engine.scene.components.media.AnimatorComponent;
import com.retro.engine.scene.components.gameplay.CharacterControllerComponent;

public class CharacterAnimatorScript extends EntityScript {
    private AnimatorComponent animator;
    private CharacterControllerComponent controller;

    @Override
    public void onStart() {
        animator = getAnimator();
        controller = getCharacterController();
    }

    @Override
    public void onUpdate(float deltaTime) {
        if (animator == null || controller == null) return;
        float speed = (float) Math.sqrt(controller.walkVelocityX * controller.walkVelocityX +
                                        controller.walkVelocityZ * controller.walkVelocityZ);
        animator.setFloat("MoveSpeed", speed);
        animator.setBool("IsGrounded", controller.isGrounded);
    }
}
```

PRODUCTION GAMEPLAY SCRIPTS — EXAMPLE 4: PHYSICS CANNON

■ PhysicsCannonScript.java

```
package com.retro.game;

import com.retro.engine.scripting.EntityScript;
import com.retro.engine.scripting.Input;
import com.retro.engine.physics.RaycastHit;
import com.retro.engine.scene.components.physics.RigidBodyComponent;
import com.retro.engine.scene.entity.Entity;
import org.lwjgl.csrf.GLFW;

public class PhysicsCannonScript extends EntityScript {
    public float impulseForce = 25.0f;
    public float maxDistance = 100.0f;

    @Override
    public void onUpdate(float deltaTime) {
        if (Input.isMouseButtonPressed(GLFW.GLFW_MOUSE_BUTTON_LEFT)) {
            float[] fwd = getForward();
            float px = getPositionX(), py = getPositionY() + 1.5f, pz = getPositionZ();

            RaycastHit hit = raycast(px, py, pz, fwd[0], fwd[1], fwd[2], maxDistance);
            if (hit.hit) {
                logInfo("Cannon hit entity: " + hit.entity.name + " at dist " + hit.distance);
                RigidBodyComponent rb = hit.entity.getComponent(RigidBodyComponent.class);
                if (rb != null) {
                    rb.applyLinearImpulse(fwd[0] * impulseForce, fwd[1] * impulseForce, fwd[2] * impulseForce);
                }
            }
        }
    }

    @Override
    public void onTriggerExit(Entity other) {
        if (other != null) {
            logInfo("Entity exited cannon safety zone: " + other.name);
        }
    }
}
```

Raycasting & Physics Interaction Mechanics

`raycast()` queries the active 3D BVH broadphase spatial tree, returning hit distance, surface normals, and the target `Entity` handle. Applying linear impulses automatically wakes up sleeping dynamic RigidBodies in `PhysicsSystem`.

6. ECS COMPONENT SCHEMA MATRICES (PART 1)

RetroEngine's Entity Component System (ECS) decouples entity identities (64-bit `EntityHandle`) from component data storage. Components are pure data containers managed in contiguous `ComponentPool` structures.

Component Class	Key @InspectorProperty Fields	Role & Engine Functionality
`Transform`	`posX, posY, posZ`, `rotX, rotY, rotZ`, `scaleX, scaleY, scaleZ`	Spatial 3D position, Euler orientation, and scale. Parent-child hierarchy matrices.
`MeshFilter`	`meshAssetGuid`, `submeshIndex`, `isShadowCaster`	Opaque reference to loaded 3D geometry mesh asset. Controls shadow casting.
`Material`	`albedoColor`, `albedoMapGuid`, `roughness`, `metallic`	Surface shading parameters, PBR texture binding handles, and tint color.
`Light`	`lightType` (DIR, POINT, SPOT), `color`, `intensity`, `range`	3D light source parameters. Shadows generated for directional lights.
`Camera`	`fov`, `nearClip`, `farClip`, `isPrimary`	Perspective 3D camera projection matrix and picture-in-picture viewport preview.
`RigidBodyComponent`	`motionType` (STATIC, KINEMATIC, DYNAMIC), `mass`, `friction`	3D physical dynamic body properties for 60 Hz physics sub-stepping.
`BoxCollider`	`sizeX, sizeY, sizeZ`, `centerOffset`, `isTrigger`	Axis-aligned / oriented box collision shape volume for broad/narrowphase.
`SphereCollider`	`radius`, `centerOffset`, `isTrigger`	Bounding sphere collision volume for fast spherical overlap checks.
`CapsuleCollider`	`radius`, `height`, `centerOffset`, `isTrigger`	Capsule collision volume ideal for character controllers and humanoids.

Zero-Allocation Component Pool Storage

Every registered ECS component is mirrored into a contiguous dense `ComponentPool` array. Systems iterate over components linearly in contiguous CPU cache lines, eliminating Java object pointer-chasing.

ECS COMPONENT SCHEMA MATRICES (PART 2)

Component Class	Key @InspectorProperty Fields	Role & Engine Functionality
`CharacterControllerComponent`	`stepHeight`, `slopeLimit`, `jumpSpeed`, `isGrounded`	FPS/TPS character movement solver with automatic stair stepping and slope limit.
`VehicleControllerComponent`	`engineForce`, `steeringAngle`, `suspensionStiffness`	4-wheel raycast vehicle physics solver with Hooke's Law spring forces.
`AudioSource`	`audioClipGuid`, `volume`, `pitch`, `isLooping`, `is3D`	OpenAL / JavaSound audio playback source with 3D spatial attenuation.
`AnimatorComponent`	`activeClipName`, `playbackSpeed`, `isPlaying`, `skeleton`	64-matrix bone palette container for zero-allocation GPU skinning.
`ScriptComponent`	`scriptPath`, `enabled`, `propertyOverrides`	Holds user `EntityScript` instance and serialized Inspector field property overrides.
`UICanvasComponent`	`renderMode` (SCREEN, WORLD), `referenceResolution`	Root 2D/3D UI canvas overlay container for HUD elements and world space UI.
`UIPanel`	`backgroundColor`, `borderRadius`, `borderWidth`	2D layout panel container supporting rounded cards and background fills.
`UIImage`	`textureGuid`, `tintColor`, `preserveAspect`	2D UI image quad renderer for HUD icons, textures, and sprites.
`UIText`	`text`, `fontGuid`, `fontSize`, `textColor`	Bitmap font text label renderer supporting wrapping and alignment.
`UIProgressBar`	`progress` (0..1), `fillColor`, `backgroundColor`	2D progress / health bar element with directional fill controls.

User Property Overrides & Reflected Fields

`ScriptComponent` parses public primitive fields on user `EntityScript` instances. When fields are modified in the Inspector, their values are stored as string key-value overrides in `ScriptComponent.propertyOverrides` and re-injected automatically whenever the script is initialized or hot-reloaded.

7. GRAPHICS HARDWARE INTERFACE (RHI) & PHYSICS SOLVERS

RetroEngine decouples high-level scene logic from graphics drivers via the `IRenderBackend` interface. The engine provides two fully featured, cross-platform RHI implementations:

Feature Dimension	Vulkan 1.3 Backend (`VulkanBackend`)	Metal 3 Backend (`MetalBackend`)
Target Platforms	Windows, Linux, macOS (via MoltenVK)	macOS (Apple Silicon & Intel Macs)
Descriptor System	Vulkan 1.2 Descriptor Indexing (Bindless Table)	Metal 3 Argument Buffers (`MTLGPUFamilyMetal3`)
Render Architecture	Dynamic Rendering (`VK_KHR_dynamic_rendering`)	Native Apple Render Pass Encoders
Shadow Rendering	2048x2048 Depth32Float Map + 3x3 PCF Filter	2048x2048 Depth32Float Map + Raycast Blobs
Shader Hot Reload	Background `glslc` SPIR-V re-compilation	Runtime `.metal` source pipeline swapping

MODERN 60 HZ PHYSICS ENGINE TOPOLOGY

The `PhysicsSystem` executes at a fixed 60 Hz step interval (0.0166s) with a maximum of 2 sub-steps per frame (`MAX_PHYSICS_STEPS = 2`) to prevent spiral-of-death catch-up loops on low-spec hardware. The physics engine is partitioned into four specialized solvers:

• **BroadphaseSolver**: Zero-allocation Dynamic AABB BVH Tree with fat AABB margin padding (`FAT_AABB_MARGIN = 0.2f`) and SAH balancing.

• **NarrowphaseSolver**: Collision shape overlap manifolds (Box, Sphere, Capsule) and positional separation impulses.

• **CharacterControllerSolver**: Vice City style character ground snapping, slope limit evaluation, and stair stepping.

• **VehicleSuspensionSolver**: 4-wheel raycast suspension solver evaluating Hooke's Law spring forces and tire friction.

8. BETA QA TESTING FIELD GUIDE & EXPECTED BEHAVIORS

As RetroEngine is currently in Beta, bug testers and developers should verify engine behavior against the expected baseline behaviors detailed below:

Subsystem Area	Expected Beta Behavior	Anomaly / Bug to Watch For
Script Hot Reload	Editing script source under `Assets/` updates running logic in <1s.	Script disabled or error logged to Console panel.
Shader Hot Reload	Modifying `.metal` / `.vert` / `.frag` files reloads pipelines cleanly.	Pipeline compilation error logged; previous pipeline kept.
Entity State Backup	Pressing Stop restores pre-play entity positions in-place.	Entity transform drift or dangling handle warnings.
Asset Streaming	Models & textures load asynchronously via Virtual Threads.	Texture placeholder white box or unreferenced VRAM leak.
Color-ID Picking	Left-clicking 3D mesh in Viewport selects entity in Hierarchy.	Clicking empty space selects background quad or wrong entity.
60 Hz Physics	Rigid bodies fall under gravity and collide with ground planes.	Bodies tunneling through thin colliders at high velocities.

Bug Reporting Protocol for Testers

When submitting bug reports, please include: (1) Host OS & GPU, (2) Engine Console Panel log output, (3) RHI backend active (Vulkan vs Metal), and (4) Exact reproduction steps starting from Project Launcher.

Summary & Next Volume

This completes Volume 1 of the RetroEngine Beta Encyclopedia. Refer to Volume 2 (Editor Suite & Viewport Workspace Guide) for complete documentation on UI DockSpace layouts, Inspector panel field reflection, 3D transform gizmos, and the skeletal animation graph state machine editor.